

دليل مشروع RAG الطبي

الإجابة المدعومة بالمصادر عن سرطان القولون والمستقيم

الفكرة في سطر واحد

NICE NG151 إلى أجزاء منظمة، نبحث فيها دلاليًا، ثم ننشئ إجابة من النصوص التي تم العثور عليها فقط مع ذكر المصدر والصفحة. نحول دليل

النطاق

- المجال: Colorectal cancer (سرطان القولون والمستقيم).
- المصدر الأساسي: دليل NICE NG151 الرسمي، وليس مواقع عشوائية أو معرفة عامة للنموذج.
- الهدف: نموذج RAG تعليمي يجيب بإسناد واضح؛ ليس بديلاً عن الطبيب أو القرار الطبي.

ماذا يغطي هذا الملف؟

والتنظيف، chunking والـ embeddings، قاعدة المتجهات والتقييم، ثم التوليد المقيد بالمصدر والإحالات. وفي النهاية توجد أسئلة جاهزة للمناقشة. الرحلة من اليوم الأول حتى اليوم الثالث: اختيار المصدر، التحويل

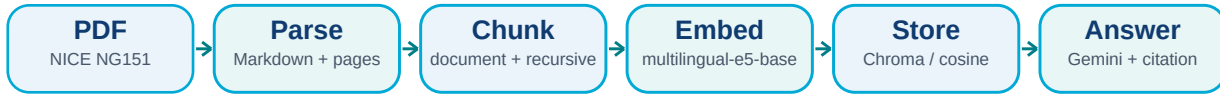
ملفات المشروع الأساسية

المسؤولية	الملف
إعدادات المسارات، النموذج، حجم الك-Top، chunk، وحد الرفض.	config.py
تحويل الـ PDF، إنشاء الـ chunks، عمل الـ embeddings، وتخزينها في Chroma.	ingest.py
استرجاع أفضل المقاطع من قاعدة المتجهات.	query.py
قياس جودة الاسترجاع على أسئلة اختبار مرجعية.	evaluate.py
ثم إجابة موثقة أو رفض آمن Gemini النهائي: استرجاع ثم RAG	generate.py

1. المعمارية الكاملة

صلة من الدليل الطبي ونمررها له. بهذا نقلل الهلوسة ونستطيع إظهار الصفحة والقسم: Retrieval-Augmented Generation اختصار RAG قبل أن يجيب النموذج، نسترجع أدلة ذات

المدخلات تتحول إلى معرفة قابلة للبحث، ثم إجابة مقيدة بالنصوص المسترجعة.



مسار التنفيذ

- PDF من NICE رسمي → Markdown نحافظ على النص والصفحات في
- نستبعد صفحات الغلاف والفهرس والمراجع الإدارية؛ نفهرس صفحات المحتوى فقط.
- نقسم المحتوى إلى chunks مع metadata مثل الصفحة والقسم ونوع المحتوى.
- نحول كل chunk إلى vector باستخدام نموذج embeddings متعدد اللغات.
- نخزن vectors و metadata في Chroma، ثم نبحث باستخدام cosine similarity.
- نرسل أفضل المقاطع فقط إلى Gemini ليصاغ الإجابة مع citation، أو يرفض إن لم تكن الأدلة كافية.

فصل مهم

يكتب الإجابة؛ وظيفته فقط تحويل النص إلى أرقام للبحث. Gemini هو نموذج التوليد الذي يصاغ الإجابة بعد أن نزوده بالمقاطع المسترجعة. لا embedding model

البيانات المهمة الناتجة

هل يدخل في التطبيق؟	ماذا يحتوي؟	المجلد/الملف
نعم، مرحلة وسيطة قابلة للمراجعة.	نسخة النص بعد التحويل من PDF مع الصفحات.	data/processed/*.md
نعم، للمراجعة وإعادة الفهرسة.	كل chunk + metadata الخاصة به.	data/chunks/*.json
نعم، هذه قاعدة البحث.	الفعلية Chroma وقاعدة vectors	data/vector_store/chroma/
للتقييم، وليس للإجابة المباشرة.	نتائج الاختبارات والأسئلة.	data/evaluation/*.csv
دليل لاختيار الإعدادات.	مقارنات chunking والنماذج.	data/experiments/

2. اليوم الأول: المصدر والتحويل

NG151 محدد مباشرة بسرطان القولون والمستقيم. اختيار مصدر موثوق أهم من جمع مصادر كثيرة غير متجانسة، خصوصا في مشروع طبي. اخترنا NICE لأنه جهة إرشادية موثوقة، والدليل

إلى نص؟ PDF كيف تحول الـ Parsing

- نستخدم pymupdf4llm لتحويل PDF إلى Markdown مع page chunks؛ لذلك لا تضيع علاقة النص برقم الصفحة.
- لا نفهرس كل PDF عميانا: صفحات الغلاف والفهرس أو المراجع الإدارية لا تقدم recommendation طبية قابلة للإجابة.
- نحفظ بالعناوين مثل section و subsection، لأن citation يحتاج اسما مفهوما للقسم وليس رقما فقط.

لماذا Markdown وليس PDF مباشرة؟

pipeline هو المادة العملية للـ Markdown يبقى المصدر الأصلي، لكن الـ PDF نص سهل الفحص والتقسيم وإعادة التشغيل عليه. الـ Markdown

مثال metadata لكل chunk

```
document: Colorectal cancer | page: 26 | section: Follow-up for detection... | content_type:
recommendation | source_url: NICE NG151
```

الـ content type

أو rationale. أثناء السؤال الطبي، البحث يفلتر إلى recommendation؛ هذا يمنع أن يسبق شرح عام أو عنوان فارغ النص الإرشادي الصريح. يتم وضع نوع للمحتوى مثل recommendation أو overview

3. Chunking: كيف قسمنا الدليل؟

وoverlap. أي أننا نبدأ من هيكل المستند (صفحة/قسم/توصية)، ثم نقسم فقط عندما تكون القطعة طويلة مع الحفاظ قدر الإمكان على الفقرات والجمل. الإجابة الجاهزة للمناقشة: نستخدم Document-based chunking مع Recursive splitting

لماذا ليس Fixed-size فقط؟

العشوائي حسب عدد الحروف قد يفصل رقم التوصية عن معناها، أو يقطع التعليمات الطبية. هيكل الدليل نفسه له قيمة: صفحة، عنوان، ورقم توصية. لأن القطع

تفاصيل الإعداد المختار

السبب	القيمة	البند
يحترم صفحة/قسم/توصية، ثم يقسم الطويل فقط.	Document-based + Recursive	النوع
سياق كاف بدون أن يصبح البحث عاما جدا.	تقريبا 500 tokens	الحجم
لا نفقد المعنى عند حدود القطع.	تقريبا 80 tokens	الoverlap
يبقى الوحدات اللغوية الطبيعية معا قدر الإمكان.	فقرات → أسطر → جمل → كلمات	الفواصل
بعد بناء الإعداد المختار على الدليل المفهرس.	103 chunks	عدد القطع

كيف تم الاختيار؟

العثور في تجربة الـ small model، لذا اخترنا 500/80 لأنه توازن عملي بين context كاف وعدد chunks معقول، بدلا من اختيار عشوائي. جربت أحجام 300/50 و 500/80 و 700/100. أعطت نفس معدل

مهم

Semantic chunking ولا Agentic chunking. لا نستدعي LLM ليقرر مكان القطع، ولا نقسم بحسب تشابه embeddings بين الجمل. هذا ليس chunking

4. الـ Vector Database و Embeddings

تمثيل رقمي لمعنى النص. النصوص ذات المعنى المتقارب تكون vectors ها أقرب رياضيا؛ لذلك يمكن البحث بسؤال عربي في دليل إنجليزي. embedding

نموذج الـ Embeddings

- الموديل النهائي: intfloat/multilingual-e5-base من Sentence Transformers.
- متعدد اللغات؛ مناسب للأسئلة العربية والإنجليزية مع source إنجليزي.
- نستخدم passage: قبل chunk عند الفهرسة، و query: قبل سؤال المستخدم؛ هذا جزء من طريقة E5 الصحيحة.
- نطبع الـ (vectors normalization) ونستخدم cosine similarity؛ الدرجة الأعلى تعني تقاربا دلاليا أكبر.

Vector database: Chroma

الأقرب، ونفلتر Top-k عند الاستعلام نسترجع persistent. في قاعدة محلية metadata مع النص والـ vector يخزن Chroma بسهولة؛ metadata filtering و persistence يوفر Chroma لأن FAISS اخترناه بدلا من content_type=recommendation. بديل ممتاز لكننا لم نستخدمه في النسخة النهائية FAISS

تسلسل سؤال واحد

Arabic question → encode_query('query: ...') → Chroma cosine search → top 5 recommendation chunks → Gemini

لماذا الاستجابة لا تبدأ من الصفر دائما؟

Python/model في أول تشغيل واستدعاء Gemini. في interactive mode تبقى النماذج في الذاكرة فتكون الأسئلة التالية أسرع. الـ vectors محفوظة مسبقا داخل Chroma؛ لا نعيد ingest أو embeddings مع كل سؤال. الذي يستهلك وقتنا هو تحميل

5. اليوم الثاني: التقييم والتحسين

لا يكفي أن يبدو جواب واحد جيدا. أنشأنا أسئلة اختبار ذات evidence متوقع ونقيس هل تم العثور على النص الصحيح ضمن Top-k.

مقاييس التقييم

- Found rate: k نسبة الأسئلة التي ظهر دليلها المتوقع داخل أفضل
- Strict Precision@k: k ؛ تنخفض طبيعيا عندما يزيد Top-k نسبة النتائج الصحيحة حرفيا داخل كل قائمة
- AR / EN found rate: قياس منفصل للأسئلة العربية والإنجليزية للتأكد من دعم اللغتين

اختيار Top-k

Top-k	Found rate	Precision@k	الفرءة
3	65%	0.3000	نتائج أدق، لكن نفوت أدلة أكثر.
4	75%	0.2562	تحسن في العثور على الدليل.
5	80%	0.2150	اختير للتوليد؛ سياق أفضل بدون إغراق كبير.
20	100%	0.0763	للتشخيص فقط؛ واسع جدا للتوليد.

مقارنة نماذج embeddings

القرارنا	الإنجليزية	العربية	Found rate	النموذج
تجربة أولى أسرع.	18/20	14/20	80.0%	multilingual-e5-small
المستخدم في النسخة النهائية.	19/20	18/20	92.5%	multilingual-e5-base

هل يجب أن تكون النتيجة 100%؟

لا تعني أن Top-20 مناسب للمستخدم؛ لأنه يدخل مقاطع كثيرة غير دقيقة. نختار Top-5 للتوازن ثم نحسن البيانات والأسئلة لاحقا. ليس شرطاً. 92.5% نتيجة جيدة للتجربة، لكن 100% في Top-20

6. اليوم الثالث: التوليد المقيد بالمصدر

خام. في `generate.py` نستخدم Gemini ليحول أفضل المقاطع إلى رد واضح، لكن داخل `guardrails` تمنعه من إضافة معلومات خارج النص. الاسترجاع وحده يعرض مقاطع

- استقبل سؤال المستخدم وحدد لغته (عربي أو إنجليزي).
- استرجع أفضل 5 recommendations من Chroma.
- إذا لم توجد نتائج أو كانت الدرجة أقل من $\text{MIN_RETRIEVAL_SCORE} = 0.60$: رد رفض آمن.
- مرر المقاطع المسترجعة فقط إلى Gemini داخل `prompt` صريح.
- تحقق من شكل الإخراج ومن أن `citation` يطابق صفحة/قسم فعليين من النتائج.
- اعرض `recommendation + excerpt + citation` باللغة المناسبة للسؤال.

نموذج التوليد

هو مولد للنص وليس `vector model`. المفتاح موجود في ملف `env` محلي ومضاف إلى `.gitignore`؛ لا يدخل في `Git` ولا في العرض. نستخدم Gemini 3.6 Flash عبر `langchain-google-genai`.

شكل الرد

سؤال عربي	سؤال انجليزي
المصدر النص الداعم التوصية	Recommendation Excerpt Citation
النص الداعم والمصدر يظهران بالعربية أيضا.	النص الداعم يبقى باللغة الأصلية من الدليل.

الرفض الآمن

نطاق الدليل، مثل أعراض سرطان المعدة، لا نخترع إجابة. نقول إن المقاطع المتاحة لا تحتوي معلومات كافية، ولا نضع `citation` مضللاً. إذا كان السؤال خارج

7. المكتبات: ما هي ولماذا تستخدم؟

المكتبة	ما هي؟	استخدامها في المشروع
pymupdf4llm	إلى Markdown بصورة مناسبة للـ LLM. أداة تحول PDF	والحفاظ على رقم الصفحة مع page chunks Parsing
langchain-text-splitters	مقسمات نص جاهزة من LangChain.	RecursiveCharacterTextSplitter للقطع الطويلة
sentence-transformers	مكتبة لتشغيل نماذج embeddings.	تحميل وتشغيل multilingual-e5-base للفهرسة والبحث.
chromadb	قاعدة vector محلية.	cosine search، embeddings، والـ metadata filtering. تخزين
langchain	إطار مساعد لبناء تطبيقات LLM.	chain. تشغيل الـ prompt وبناء text splitter
langchain-google-genai	LangChain بنماذج Google Gemini ربط	استدعاء Gemini لصياغة الرد النهائي.
python-dotenv	تحميل متغيرات البيئة من ملف محلي.	قراءة GEMINI_API_KEY بأمان من .env.

تنبيه عن مكتبات لم نستخدمها

؛ اختيارنا الحالي هو PDF parsing بديل للـ LlamaParse و Chroma بديل لـ FAISS. لم يدخلنا النسخة النهائية LlamaParse و FAISS المطلوبة metadata لأنه أبسط محليا ويحافظ على Chroma + pymupdf4llm

8. أهم أجزاء الكود

نقطة مهمة عند الشرح	يفعل ماذا؟	الجزء
أو Top-k أو chunk size يكون هنا، لا داخل عدة ملفات. تغيير model	مكان واحد للإعدادات.	config.py
يحافظ على page number ليستعمل في citation.	يحول الدليل إلى Markdown + pages.	ingest.py: parse_pdf
أولا حسب توصية/صفحة، ثم Recursive للقطعة الكبيرة.	ينشئ chunks منظمة.	ingest.py: split_page
prefix: passage:.. يحتاج E5	يحول chunks إلى vectors.	encode_document
recommndation. وفلتر prefix: query: يحتاج E5	يحول السؤال إلى vector ويسترجع Top-k.	query.py: search
نقيس retrieval، لا جودة صياغة Gemini وحدها.	يقارن النتائج بـ expected evidences.	evaluate.py
لا يسمح للنموذج باستخدام معرفة خارج الـ passages.	citation ثم prompt ثم retrieval validation.	generate.py

التشغيل الأساسي

```
python ingest.py python evaluate.py --top-k 5 python generate.py "What follow-up is recommended after curative colorectal cancer surgery?" python generate.py --interactive
```

الأداء

الـ embeddings لتجنب محاولة الاتصال بـ Hugging Face. في تطبيق ويب يظل الـ process شغالا؛ لذلك لا يعاد تحميل النموذج مع كل سؤال. الأول للموديل طبيعي أن يأخذ وقتا. تم استخدام caching بـ lru_cache في query.py و generate.py، ووضع local_files_only لنموذج التحميل

9. أسئلة متوقعة في المناقشة

السؤال	إجابة مختصرة جاهزة
ما المشكلة التي يحلها المشروع؟	يوفر إجابة عن الدليل الطبي مع الدليل والصفحة بدلا من إجابة عامة بلا مصدر.
لماذا NICE NG151؟	جهة إرشادية موثوقة، والدليل متخصص ومحدد في سرطان القولون والمستقيم.
ما هو RAG؟	استرجاع نصوص موثوقة أولا ثم توليد الإجابة بالاعتماد عليها.
ما نوع chunking؟	؛ نبدأ بالصفحة والقسم والتوصية Recursive splitting مع Document-based
ما المقصود بالoverlap؟	جزء صغير مشترك بين chunks متجاورة حتى لا يضيع السياق عند الحدود.
لماذا E5-base؟	أفضل نتيجة في التجربة: 92.5% Found rate ودعم أقوى للعربية والإنجليزية.
لماذا نضيف query: و passage؟	هذه صيغة E5 لتوضيح دور النص وتحسين جودة الاسترجاع.
لماذا Chroma وليس FAISS؟	بديل لم نحتجها هنا FAISS بسهولة؛ metadata filtering يحفظ البيانات محليا ويدعم Chroma
لماذا Top-k = 5؟	توازن عملي: يجلب أدلة أكثر من 3 أو 4، وأقل ضوضاء من 20.
ما معنى 92.5%؟	في 37 من 40 سؤالا داخل النطاق، ظهر الدليل المتوقع ضمن أفضل 5 نتائج.
كيف تمنعون hallucination؟	رفض عندما لا توجد أدلة كافية + citation validation + مقيد بالمقاطع المسترجعة فقط Prompt
هل النظام تشخيص طبي؟	لا. هو أداة استرجاع وشرح من guideline، ولا يستبدل الطبيب أو القرار السريري.

الخلاصة

محفظة السياق → embeddings متعددة اللغات → Chroma → تقييم → Gemini مقيد بالمصدر مع citations ورفض آمن.
لدينا pipeline قابل للتكرار: مصدر رسمي → نص منظم → chunks

مصدر الدليل: NICE. Colorectal cancer (NG151): <https://www.nice.org.uk/guidance/ng151>